# JobKit - Teradata DBA interview questions

212 questions for Teradata DBA. Each answer is five pointers with working code. Single-file download.

### 1. What is Teradata?

**1. Definition you should say first: A shared-nothing MPP database for large analytic workloads. Then spend the rest of the answer on architecture, a working example, and a production failure.**
**2. Teradata is a shared-nothing MPP analytic database. Vantage is the current platform branding (SQL plus analytics). Each AMP owns a slice of every table.**
**3. Draw PE -> BYNET -> AMPs. Mention PI, stats, TASM, and BAR in the first minute.**
**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.DiskSpaceV WHERE DatabaseName = 'EDW';
HELP SESSION;
SELECT HASHAMP() AS amps;
```

**5. Calling it 'just like Oracle but faster' without AMP/PI/skew will not survive a DBA round.**

### 2. What is an AMP?

**1. Definition you should say first: Access Module Processor: a parallel worker that stores and processes part of the data. Then spend the rest of the answer on architecture, a working example, and a production failure.**
**2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.**
**3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.**
**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

**5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.**

### 3. What is a PE?

**1. Definition you should say first: Parsing Engine: parses SQL, optimizes, and sends steps to AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.**
**2. PE = Parsing Engine: session handling, parse, optimize, dispatch steps to AMPs.**
**3. PE CPU spikes on bad dynamic SQL or too many short tactical requests. TASM protects mixed workloads.**
**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT AccountName, UserName FROM DBC.SessionInfoV WHERE UserName = USER;
EXPLAIN SELECT COUNT(*) FROM edw.fact_orders;
```

**5. People confuse PE with AMP. PE does not store the table; AMPs do.**

### 4. What is BYNET?

**1. Definition you should say first: The interconnect that moves messages and data between PEs and AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.**
**2. BYNET is the interconnect for messages, row redistribution, and duplication between PEs and AMPs.**
**3. Product joins and redistributing a huge uncompressed table flood BYNET. Check ResUsage and Viewpoint fabric metrics.**
**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT *
FROM edw.fact_orders f
JOIN edw.dim_customer d ON d.region = f.region;
-- look for: redistributing / duplicating all rows of
```

**5. If you only add CPU and ignore join geography, BYNET will still stall.**

### 5. What is a vproc?

**1. Definition you should say first: A virtual processor such as AMP or PE running on a node. Then spend the rest of the answer on architecture, a working example, and a production failure.**
**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**
**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**
**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
```

```
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 6. What is a clique?

**1. Definition you should say first: A set of nodes that share disks so a failed node can fail over. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 7. What is a hot standby node?

**1. Definition you should say first: A spare node that can take over vprocs after a failure. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 8. What is Fallback?

**1. Definition you should say first: A second copy of a table on another AMP for availability. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Fallback stores a second copy of rows on another AMP so a down AMP does not lose the table.**

**3. Enable on critical dimensions and control tables. It costs disk and write I/O. Not a substitute for BAR.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_calendar (
  cal_dt DATE NOT NULL
)
PRIMARY INDEX (cal_dt)
FALLBACK;
```

**5. Fallback does not protect against a bad UPDATE. You still need backups and a back-out plan.**

## 9. When do you use Fallback?

**1. Definition you should say first: Critical tables that must survive AMP or disk loss. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Fallback stores a second copy of rows on another AMP so a down AMP does not lose the table.**

**3. Enable on critical dimensions and control tables. It costs disk and write I/O. Not a substitute for BAR.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_calendar (
  cal_dt DATE NOT NULL
)
PRIMARY INDEX (cal_dt)
FALLBACK;
```

**5. Fallback does not protect against a bad UPDATE. You still need backups and a back-out plan.**

## 10. What is a Primary Index?

**1. Definition you should say first: The distribution key that decides which AMP stores a row. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.**

**3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

**5. Changing PI means a full rebuild. Get it right on day one of the model.**

## 11. Unique vs non-unique PI?

**1. Definition you should say first: UPI enforces uniqueness; NUPI allows duplicates and can skew. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.**

**3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

**5. Changing PI means a full rebuild. Get it right on day one of the model.**

## 12. What is a Secondary Index?

**1. Definition you should say first: An extra access path; it has a subtable and extra I/O. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.**

**3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

**5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.**

## 13. What is a Join Index?

**1. Definition you should say first: A stored join or aggregation that the optimizer can use. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.**

**3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

**5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.**

## 14. What is PPI?

**1. Definition you should say first: Partitioned Primary Index: partitions rows inside each AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.**

**3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE edw.fact_orders (
```

```
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

**5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.**

## 15. What is a NoPI table?

**1. Definition you should say first: A table without a PI; used for fast staging loads. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.**

**3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

**5. Changing PI means a full rebuild. Get it right on day one of the model.**

## 16. What is a sparse map?

**1. Definition you should say first: A map that uses a subset of AMPs for smaller tables. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.**

**3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

**5. Reconfig without a back-out window is a career incident.**

## 17. What is a contiguous map?

**1. Definition you should say first: The default map that uses all AMPs in the system. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.**

**3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

**5. Reconfig without a back-out window is a career incident.**

## 18. What is hashing in Teradata?

**1. Definition you should say first: The PI value is hashed to pick the AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.**

**3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE stg.orders_in (
```

```
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

**5. Changing PI means a full rebuild. Get it right on day one of the model.**

## 19. What is skew?

**1. Definition you should say first: Uneven row or CPU distribution across AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Skew is uneven rows or CPU across AMPs. Low-cardinality PI (country, flag, null-heavy key) causes it.**

**3. Compare AMP row counts, DBQL, Viewpoint skew. Fix PI, add a composite PI, or stage and redistribute on a better key.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW(pi_col))) AS amp_n, COUNT(*) AS rows
FROM edw.fact_orders
GROUP BY 1
ORDER BY 2 DESC;
-- skew ratio = max/avg
```

**5. If NULLs hash together, a nullable PI becomes one giant AMP. Coalesce or pick another key.**

## 20. How do you find skewed tables?

**1. Definition you should say first: Compare AMP row counts or use DBQL and Viewpoint. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Skew is uneven rows or CPU across AMPs. Low-cardinality PI (country, flag, null-heavy key) causes it.**

**3. Compare AMP row counts, DBQL, Viewpoint skew. Fix PI, add a composite PI, or stage and redistribute on a better key.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW(pi_col))) AS amp_n, COUNT(*) AS rows
FROM edw.fact_orders
GROUP BY 1
ORDER BY 2 DESC;
-- skew ratio = max/avg
```

**5. If NULLs hash together, a nullable PI becomes one giant AMP. Coalesce or pick another key.**

## 21. What is a hot AMP?

**1. Definition you should say first: An AMP doing more work than others, often from skew. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.**

**3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

**5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.**

## 22. What is spool space?

**1. Definition you should say first: Temporary space for intermediate query results. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

## 23. What is perm space?

**1. Definition you should say first: Permanent table storage quota. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

### 24. What is temp space?

**1. Definition you should say first: Space for global temporary tables. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

### 25. User vs database in Teradata?

**1. Definition you should say first: A user can log on; a database is a space container. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

### 26. What is a profile in Teradata?

**1. Definition you should say first: A set of account, spool, and password settings for users. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

### 27. What is a role?

**1. Definition you should say first: A bundle of rights you grant to users. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
```

```
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 28. What is GRANT vs role?

**1. Definition you should say first: GRANT is a privilege; a role groups many privileges. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 29. What is a macro?

**1. Definition you should say first: A stored SQL snippet executed with EXEC. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 30. What is a stored procedure?

**1. Definition you should say first: Procedural SQL with logic, not just a macro. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 31. What is a volatile table?

**1. Definition you should say first: Session-scoped table that disappears on logoff. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
```

```
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 32. Global temporary vs volatile?

**1. Definition you should say first: GTT definition is permanent; data is session-scoped. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 33. What is a derived table?

**1. Definition you should say first: An inline subquery in the FROM clause. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 34. What is a CTE?

**1. Definition you should say first: WITH clause query that names a result set. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 35. What is COLLECT STATISTICS?

**1. Definition you should say first: Gather data demographics so the optimizer can plan well. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

### 36. When do you collect stats?

**1. Definition you should say first: After large loads, skew, or bad plans; not blindly every hour. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

### 37. What is SUMMARY stats?

**1. Definition you should say first: A cheaper stats option on some releases for quick demographics. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

### 38. What is sampled stats?

**1. Definition you should say first: Stats on a sample; faster but less accurate. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

### 39. How do you see a plan?

**1. Definition you should say first: EXPLAIN SELECT ... Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 40. What is a confidence level in EXPLAIN?

**1. Definition you should say first: How sure the optimizer is about row estimates. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 41. What is a product join?

**1. Definition you should say first: Every row compared to every row; often a warning sign. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 42. What is a merge join?

**1. Definition you should say first: Join on sorted/hashed keys; usually cheaper. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 43. What is a hash join?

**1. Definition you should say first: Build a hash table of the smaller set and probe it. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 44. What is a nested join?

**1. Definition you should say first: Index-driven lookup join. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 45. What is redistribution?

**1. Definition you should say first: Moving rows across AMPs so join keys co-locate. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 46. What is duplication of a table in a join?

**1. Definition you should say first: Copying a small table to all AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 47. What is a residual condition?

**1. Definition you should say first: A filter applied after the join or retrieve step. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.**

**3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

**5. SET tables with duplicate checks plus a product join will run until spool dies.**

## 48. What is a lock in Teradata?

**1. Definition you should say first: Access, read, write, or exclusive lock on a table or row hash. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

### 49. What is a deadlock?

**1. Definition you should say first: Two sessions waiting on each other; one is aborted. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

### 50. How do you reduce lock contention?

**1. Definition you should say first: Shorter transactions, nightly batch windows, and row-hash locks. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

### 51. What is a transaction in Teradata?

**1. Definition you should say first: BT/ET or ANSI transaction around SQL. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BTET (Teradata mode) auto-commits each request unless BT/ET. ANSI uses BEGIN/COMMIT. A request is one SQL; a transaction can be many.**

**3. ETL scripts should be explicit: BT; steps; ET; with error handling in BTEQ.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.SET SESSION TRANSACTION BTET
BT;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-01';
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
ET;
```

**5. A failed INSERT after DELETE without a transaction leaves an empty partition. That is a Sev1.**

### 52. BTET vs ANSI mode?

**1. Definition you should say first: BTET is Teradata transaction mode; ANSI is commit-oriented. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BTET (Teradata mode) auto-commits each request unless BT/ET. ANSI uses BEGIN/COMMIT. A request is one SQL; a transaction can be many.**

**3. ETL scripts should be explicit: BT; steps; ET; with error handling in BTEQ.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.SET SESSION TRANSACTION BTET
BT;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-01';
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
ET;
```

**5. A failed INSERT after DELETE without a transaction leaves an empty partition. That is a Sev1.**

## 53. What is FastLoad?

**1. Definition you should say first: Utility to load empty tables quickly with two phases. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 54. FastLoad limitation?

**1. Definition you should say first: Target must be empty; no SI during load in classic use. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 55. What is MultiLoad?

**1. Definition you should say first: Utility for insert/update/delete on populated tables. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 56. What is TPump?

**1. Definition you should say first: Low-volume continuous load with row-hash locks. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 57. What is FastExport?

**1. Definition you should say first: High-volume export from Teradata. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 58. What is BTEQ?

**1. Definition you should say first: CLI for SQL, scripts, and simple imports. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 59. What is TPT?

**1. Definition you should say first: Teradata Parallel Transporter: modern load/export framework. Then spend the rest of the**

**answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 60. TPT vs legacy utilities?

**1. Definition you should say first: TPT can replace FastLoad/MultiLoad/FastExport with operators. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 61. What is a load slot?

**1. Definition you should say first: A limit on concurrent load jobs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 62. What is a checkpoint in MultiLoad?

**1. Definition you should say first: A restart point after a failure. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 63. What is an error table?

**1. Definition you should say first: Where utilities land rejected rows. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 64. What is a UV table?

**1. Definition you should say first: Uniqueness violation table in MultiLoad. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 65. What is Viewpoint?

**1. Definition you should say first: Web console for health, queries, and alerts. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters,**

**exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 66. What is QueryGrid?

**1. Definition you should say first: Fabric to query Hadoop, Oracle, and other systems from Teradata. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 67. What is TASM?

**1. Definition you should say first: Teradata Active System Management: workload rules and throttles. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 68. What is a workload in TASM?

**1. Definition you should say first: A class of queries with priority and limits. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 69. What is a throttle?

**1. Definition you should say first: A cap on concurrency for a workload. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 70. What is a filter in TASM?

**1. Definition you should say first: A rule that rejects or delays some queries. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 71. What is exception handling in TASM?

**1. Definition you should say first: Move or abort queries that exceed CPU or skew. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 72. What is WD-Default?

**1. Definition you should say first: The default workload when nothing else matches. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 73. What is a utility session?

**1. Definition you should say first: Sessions used by load/export utilities. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 74. What is BAR?

**1. Definition you should say first: Backup and restore, often with DSA or NetBackup. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
```

```
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 75. What is DSA?

**1. Definition you should say first: Data Stream Architecture for backups. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 76. Full vs incremental backup?

**1. Definition you should say first: Full copies everything; incremental copies changes. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 77. How do you restore one table?

**1. Definition you should say first: Use BAR object-level restore if the backup supports it. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 78. What is a dump vs backup?

**1. Definition you should say first: Dump often means a system dump for support; backup is data protection. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 79. What is PDBA vs production DBA?

**1. Definition you should say first: PDBA may own platform; production DBA owns day-to-day jobs and users. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 80. What is space reclamation?

**1. Definition you should say first: Freeing perm after deletes; packing and fallback copies matter. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

## 81. What is a cylinder?

**1. Definition you should say first: A unit of disk allocation on an AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 82. What is a table header?

**1. Definition you should say first: Metadata stored with the table on each AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 83. What is a journal?

**1. Definition you should say first: Before/after image used for recovery on some tables. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 84. Permanent journal vs transient?

**1. Definition you should say first: Permanent is extra protection; transient is for transactions. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 85. What is TVS?

**1. Definition you should say first: Teradata Virtual Storage: storage virtualization layer. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 86. What is FSYS?

**1. Definition you should say first: File system layer under AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 87. What is a node crash recovery?

**1. Definition you should say first: Vprocs migrate; cliques and HSN determine downtime. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 88. What is a database restart?

**1. Definition you should say first: TPA reset: sessions drop; need to reconnect. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 89. What is TPA?

**1. Definition you should say first: Teradata parallel database process stack. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 90. What is PDE?

**1. Definition you should say first: Parallel Database Extensions: OS layer for vprocs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 91. What is a gateway?

**1. Definition you should say first: Connection manager for client sessions. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 92. What is a session?

**1. Definition you should say first: A logged-on PE connection from a client. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 93. Default session mode?

**1. Definition you should say first: Depends on client; know if you are ANSI or Teradata mode. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 94. What is a request vs a transaction?

**1. Definition you should say first: A request is one SQL; a transaction may be many requests. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BTET (Teradata mode) auto-commits each request unless BT/ET. ANSI uses BEGIN/COMMIT. A request is one SQL; a transaction can be many.**

**3. ETL scripts should be explicit: BT; steps; ET; with error handling in BTEQ.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.SET SESSION TRANSACTION BTET
BT;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-01';
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
ET;
```

**5. A failed INSERT after DELETE without a transaction leaves an empty partition. That is a Sev1.**

## 95. How do you kill a bad query?

**1. Definition you should say first: Abort in Viewpoint or with a console abort after checking impact. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 96. How do you find who filled the spool?

**1. Definition you should say first: DBQL plus user spool limits. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

## 97. What is DBQL?

**1. Definition you should say first: Database Query Log: history of queries and steps. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 98. What is ResUsage?

**1. Definition you should say first: Resource usage macros for CPU, disk, and BYNET. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
```

```
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 99. What is a heart beat in Viewpoint?

**1. Definition you should say first: System health pulse; red means investigate now. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 100. What is a blocked session?

**1. Definition you should say first: Waiting on a lock held by another session. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

## 101. How do you see blockers?

**1. Definition you should say first: Lock viewer in Viewpoint or lock query views. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

## 102. What is a deadlock timeout?

**1. Definition you should say first: How long Teradata waits before aborting a deadlock victim. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

## 103. What is a PPI advantage?

**1. Definition you should say first: Partition elimination so fewer cylinders are read. Then spend the rest of the answer on**

**architecture, a working example, and a production failure.**

**2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.**

**3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

**5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.**

## 104. What is a character PPI?

**1. Definition you should say first: Partitioning on a character column; less common than date PPI. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.**

**3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

**5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.**

## 105. What is multilevel PPI?

**1. Definition you should say first: More than one partition expression. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.**

**3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

**5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.**

## 106. What is a column partition (CP)?

**1. Definition you should say first: Columnar storage for analytic tables on some releases. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.**

**3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
```

```
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

**5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.**

## 107. What is Hybrid Row/Column?

**1. Definition you should say first: Mix of row and column partitions. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.**

**3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

**5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.**

## 108. What is compression in Teradata?

**1. Definition you should say first: MVC, UDT, algorithmic, or block-level depending on release. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.**

**3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

**5. Compressing a near-unique column wastes CPU for no disk win.**

## 109. What is MVC?

**1. Definition you should say first: Multi-Value Compression: dictionary compression on a column. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.**

**3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

**5. Compressing a near-unique column wastes CPU for no disk win.**

## 110. What is a UDT?

**1. Definition you should say first: User-defined type. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.**

**3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

**5. Compressing a near-unique column wastes CPU for no disk win.**

## 111. What is a UDF?

**1. Definition you should say first: User-defined function. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.**

**3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

**5. Compressing a near-unique column wastes CPU for no disk win.**

## 112. What is a table operator?

**1. Definition you should say first: SQL-HMR style operator for custom processing. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.**

**3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

**5. Compressing a near-unique column wastes CPU for no disk win.**

## 113. What is Native Object Store (NOS)?

**1. Definition you should say first: Read/write object storage (S3-style) from Teradata Vantage. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.**

**3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

**5. Compressing a near-unique column wastes CPU for no disk win.**

## 114. What is Vantage?

**1. Definition you should say first: Teradata's analytics platform branding for newer releases. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata is a shared-nothing MPP analytic database. Vantage is the current platform branding (SQL plus analytics). Each AMP owns a slice of every table.**

**3. Draw PE -> BYNET -> AMPs. Mention PI, stats, TASM, and BAR in the first minute.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.DiskSpaceV WHERE DatabaseName = 'EDW';
HELP SESSION;
SELECT HASHAMP() AS amps;
```

**5. Calling it 'just like Oracle but faster' without AMP/PI/skew will not survive a DBA round.**

## 115. What is QueryBand?

**1. Definition you should say first: Name/value tags on a session or transaction for workload and audit. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 116. What is an account string?

**1. Definition you should say first: Account used for chargeback and TASM mapping. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 117. What is a performance group?

**1. Definition you should say first: Older priority scheme; TASM workloads replaced much of this. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 118. What is a resource partition?

**1. Definition you should say first: A slice of CPU reserved for a set of workloads. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 119. What is tactical vs decision support?

**1. Definition you should say first: Tactical is short; DSS is heavy reporting. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 120. How do you protect tactical queries?

**1. Definition you should say first: TASM: high priority, low throttle, and exception on long CPU. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 121. What is a delayed query?

**1. Definition you should say first: Queued by a throttle until a slot is free. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 122. What is a reject in TASM?

**1. Definition you should say first: The query is not allowed to run. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 123. What is an object lock vs row hash lock?

**1. Definition you should say first: Object is table-level; row hash is finer. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
```

```
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

## 124. What is a dummy row lock?

**1. Definition you should say first: Locking a hash with no row; still blocks that hash. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

## 125. How do you load a large fact table daily?

**1. Definition you should say first: Stage NoPI or FastLoad, then INSERT-SELECT with collect stats. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata is shared-nothing MPP: PE parses, BYNET ships, AMPs own their rows and spool.**

**3. Always name PI, stats, skew, spool, and the utility you would use. Then EXPLAIN before you run in prod.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT * FROM edw.fact_orders
WHERE order_dt = DATE '2026-09-01';
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id, order_dt);
```

**5. A product join plus missing stats plus a bad NUPI will light up one hot AMP and fill spool.**

## 126. What is an INSERT-SELECT?

**1. Definition you should say first: Load from one table into another set-based. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 127. What is MERGE INTO?

**1. Definition you should say first: Upsert based on a match condition. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
```

```
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 128. What is UPDATE from a join?

**1. Definition you should say first: Set-based update using another table. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 129. What is a SET vs MULTISET table?

**1. Definition you should say first: SET rejects duplicate rows; MULTISET allows them. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SET tables reject duplicate rows (expensive duplicate row check). MULTISET allows duplicates and is the default for most ETL facts.**

**3. Use SET only when uniqueness must be enforced by the table and volume is modest. Prefer UPI or USI + MULTISET for facts.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE SET TABLE edw.dim_code (
  code CHAR(4) NOT NULL,
  descr VARCHAR(40)
) UNIQUE PRIMARY INDEX (code);
CREATE MULTISET TABLE edw.fact_click (click_id BIGINT, url VARCHAR(200)) PRIMARY INDEX (click_id);
```

**5. A SET fact with no UPI will spend CPU comparing rows on every insert.**

## 130. Duplicate row check cost?

**1. Definition you should say first: SET tables can be expensive on NUPI during inserts. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SET tables reject duplicate rows (expensive duplicate row check). MULTISET allows duplicates and is the default for most ETL facts.**

**3. Use SET only when uniqueness must be enforced by the table and volume is modest. Prefer UPI or USI + MULTISET for facts.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE SET TABLE edw.dim_code (
  code CHAR(4) NOT NULL,
  descr VARCHAR(40)
) UNIQUE PRIMARY INDEX (code);
CREATE MULTISET TABLE edw.fact_click (click_id BIGINT, url VARCHAR(200)) PRIMARY INDEX (click_id);
```

**5. A SET fact with no UPI will spend CPU comparing rows on every insert.**

## 131. What is a queue table?

**1. Definition you should say first: A table with consume semantics for messaging-style pulls. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 132. What is an identity column?

**1. Definition you should say first: System-generated numbers; be careful with distribution. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.**

**3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

**5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.**

## 133. What is a date PI problem?

**1. Definition you should say first: Low cardinality dates skew if used as PI. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.**

**3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

**5. Changing PI means a full rebuild. Get it right on day one of the model.**

## 134. Good PI for a fact table?

**1. Definition you should say first: A high-cardinality join key, often a customer or transaction id. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.**

**3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
```

```
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

**5. Changing PI means a full rebuild. Get it right on day one of the model.**

## 135. What is referential integrity in Teradata?

**1. Definition you should say first: Optional; soft RI is often used for optimizer hints. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 136. What is a batch vs tactical window?

**1. Definition you should say first: Batch at night with loads; tactical during business hours. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata is shared-nothing MPP: PE parses, BYNET ships, AMPs own their rows and spool.**

**3. Always name PI, stats, skew, spool, and the utility you would use. Then EXPLAIN before you run in prod.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT * FROM edw.fact_orders
WHERE order_dt = DATE '2026-09-01';
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id, order_dt);
```

**5. A product join plus missing stats plus a bad NUPI will light up one hot AMP and fill spool.**

## 137. How do you handle a failed FastLoad?

**1. Definition you should say first: Drop or empty the target, fix the file, restart; check error tables. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 138. What is a two-phase FastLoad?

**1. Definition you should say first: Phase 1 loading to AMPs; phase 2 sorting into the table. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework.**

**INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 139. Can FastLoad load a PI with SI?

**1. Definition you should say first: Classic FastLoad wants no secondary indexes on the target. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 140. What is a MiniBatch?

**1. Definition you should say first: A pattern of small MultiLoad or TPT jobs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 141. What is streaming ingest?

**1. Definition you should say first: TPump or TPT Stream for near-real-time. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.**

**3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.**

**4. Working code / syntax (write this on Teradata SQL):**

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

**5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.**

## 142. How do you grant access to a new analyst?

**1. Definition you should say first: Role-based: create user, grant role, set spool and profile. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 143. How do you revoke a leaver?

**1. Definition you should say first: Lock the user, revoke roles, drop or keep for audit per policy. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 144. What is a password lockout?

**1. Definition you should say first: Profile setting after failed logons. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 145. What is LDAP vs TD database auth?

**1. Definition you should say first: Enterprise directory vs Teradata passwords. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
```

```
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 146. What is SSO to Viewpoint?

**1. Definition you should say first: Viewpoint can use corporate identity; still map TD users. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.**

**3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

**5. Personal copies of prod tables in a sandbox with PII is an audit finding.**

## 147. What is a zone in Viewpoint?

**1. Definition you should say first: A filtered view of systems for a team. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 148. What is a portlet?

**1. Definition you should say first: A Viewpoint dashboard widget. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.**

**3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

**5. Everything in WD-Default means TASM is not actually implemented.**

## 149. What is a system heartbeat alert?

**1. Definition you should say first: Pager-worthy if the database is down. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata is shared-nothing MPP: PE parses, BYNET ships, AMPs own their rows and spool.**

**3. Always name PI, stats, skew, spool, and the utility you would use. Then EXPLAIN before you run in prod.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT * FROM edw.fact_orders
WHERE order_dt = DATE '2026-09-01';
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id, order_dt);
```

**5. A product join plus missing stats plus a bad NUPI will light up one hot AMP and fill spool.**

## 150. How do you plan an upgrade?

**1. Definition you should say first: Read the release, test BAR, check views, schedule an outage window. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Upgrades need compat checks, BAR, a freeze, and a back-out. Stall in Viewpoint is wait time, not always a down system.**

**3. Read release notes for TPT/TASM changes. Run CHECKTABLE only per vendor guidance.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- pre-upgrade
-- 1 BAR full  2 capture TASM  3 freeze  4 upgrade  5 validate Viewpoint heartbeat  6 smoke FastLoad + SELECT HASHAMP()
```

**5. Skipping a backup because 'it is only a patch' is the story in half of DBA incident talks.**

## 151. What is a compat view?

**1. Definition you should say first: Older dictionary views kept for scripts. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.**

**3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

**5. Updating DBC base tables by hand is how you corrupt a system.**

## 152. DBC vs SYSLIB?

**1. Definition you should say first: DBC is the data dictionary; SYSLIB holds functions. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.**

**3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

**5. Updating DBC base tables by hand is how you corrupt a system.**

## 153. What is DBC.TablesV?

**1. Definition you should say first: Dictionary view of tables. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.**

**3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

**5. Updating DBC base tables by hand is how you corrupt a system.**

## 154. How do you find table size?

**1. Definition you should say first: SUM of currentperm from DBC.TableSizeV grouped by table. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.**

**3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
```

```
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

**5. Updating DBC base tables by hand is how you corrupt a system.**

## 155. How do you find user space?

**1. Definition you should say first: DBC.DiskSpaceV or Viewpoint space portlet. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.**

**3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

**5. Updating DBC base tables by hand is how you corrupt a system.**

## 156. What is currentperm vs peakperm?

**1. Definition you should say first: Current used vs high-water used. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.**

**3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

**5. Granting huge spool to hide a product join is not tuning.**

## 157. What is a stall?

**1. Definition you should say first: AMPs waiting on I/O or each other; check ResUsage. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 158. What is a bottleneck on BYNET?

**1. Definition you should say first: Heavy redistribution; redesign PI or join. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BYNET is the interconnect for messages, row redistribution, and duplication between PEs and AMPs.**

**3. Product joins and redistributing a huge uncompressed table flood BYNET. Check ResUsage and Viewpoint fabric metrics.**

**4. Working code / syntax (write this on Teradata SQL):**

```
EXPLAIN
SELECT *
FROM edw.fact_orders f
JOIN edw.dim_customer d ON d.region = f.region;
-- look for: redistributing / duplicating all rows of
```

**5. If you only add CPU and ignore join geography, BYNET will still stall.**

## 159. What is a CPU-bound query?

**1. Definition you should say first: High AMP CPU; check product joins and functions on columns. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.**

**3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

**5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.**

## 160. What is a missing stats symptom?

**1. Definition you should say first: Wrong join type and huge spool. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

## 161. How often to collect stats on a large fact?

**1. Definition you should say first: After major loads; use THRESHOLD where available. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

## 162. What is stats THRESHOLD?

**1. Definition you should say first: Skip collect if data has not changed enough. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.**

**3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.**

**4. Working code / syntax (write this on Teradata SQL):**

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

**5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.**

## 163. What is a copy job in BAR?

**1. Definition you should say first: Copy backup data to another system. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 164. What is a restore to a different system?

**1. Definition you should say first: Needs compatible maps and enough AMPs or a remap strategy. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.**

**3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

**5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.**

## 165. What is a map change?

**1. Definition you should say first: Redistribute tables after adding AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.**

**3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

**5. Reconfig without a back-out window is a career incident.**

## 166. What is a reconfig?

**1. Definition you should say first: System change such as adding nodes; planned outage. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.**

**3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

**5. Reconfig without a back-out window is a career incident.**

## 167. What is HSN vs clique failover?

**1. Definition you should say first: HSN is a spare node; clique shares disks among nodes. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 168. What is a down AMP?

**1. Definition you should say first: AMP not processing; fallback or HSN may cover. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.**

**3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
```

```
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

**5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.**

## 169. What is a catchup?

**1. Definition you should say first: Recovering fallback copies after a component returns. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.**

**3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.**

**4. Working code / syntax (write this on Teradata SQL):**

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

**5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.**

## 170. What is a checktable?

**1. Definition you should say first: Utility to check table integrity. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 171. What is a scandisk?

**1. Definition you should say first: Lower-level disk integrity check. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

## 172. What is a perm cache?

**1. Definition you should say first: Memory cache of perm data; watch hit rates. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PE = Parsing Engine: session handling, parse, optimize, dispatch steps to AMPs.**

**3. PE CPU spikes on bad dynamic SQL or too many short tactical requests. TASM protects mixed workloads.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT AccountName, UserName FROM DBC.SessionInfoV WHERE UserName = USER;
EXPLAIN SELECT COUNT(*) FROM edw.fact_orders;
```

**5. People confuse PE with AMP. PE does not store the table; AMPs do.**

## 173. What is FSG cache?

**1. Definition you should say first: File segment cache for I/O. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

### 174. What is a TVS temperature?

**1. Definition you should say first: Hot/cold data placement on mixed storage. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.**

**3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

**5. Running checktable on a live 100 TB fact without a window can be an outage.**

### 175. What is a 1-AMP operation?

**1. Definition you should say first: A query that hits a single AMP via unique PI access. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.**

**3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

**5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.**

### 176. What is an all-AMP retrieve?

**1. Definition you should say first: Every AMP reads its piece of the table. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.**

**3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

**5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.**

### 177. What is a group AMP operation?

**1. Definition you should say first: A subset of AMPs, for example sparse maps. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.**

**3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.**

**4. Working code / syntax (write this on Teradata SQL):**

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

**5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.**

### 178. What is a join index maintenance cost?

**1. Definition you should say first: Extra work on every base-table change. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.**

**3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

**5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.**

## 179. When is a JI worth it?

**1. Definition you should say first: Stable, repeated joins that dominate CPU. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.**

**3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

**5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.**

## 180. What is an aggregate JI?

**1. Definition you should say first: Pre-aggregated stored result. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.**

**3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

**5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.**

## 181. What is a single-table JI?

**1. Definition you should say first: A stored projection or extra access path on one table. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.**

**3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

**5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.**

## 182. What is a covering index?

**1. Definition you should say first: An index that satisfies a query without the base table. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.**

**3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

**5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.**

## 183. What is a value-ordered NUSI?

**1. Definition you should say first: NUSI stored in value order for range access. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.**

**3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

**5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.**

## 184. What is a hash-ordered NUSI?

**1. Definition you should say first: Default NUSI organization. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.**

**3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

**5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.**

## 185. What is a unique secondary index?

**1. Definition you should say first: Enforces uniqueness and supports lookups. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.**

**3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

**5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.**

## 186. Cost of USI on load?

**1. Definition you should say first: Load utilities may require dropping USI first. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.**

**3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

**5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.**

## 187. What is a soft RI?

**1. Definition you should say first: Optimizer uses the relationship without enforcing it. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 188. What is a batch error table?

**1. Definition you should say first: Landing for failed rows in a batch. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
```

```
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 189. How do you handle late-arriving dimensions?

**1. Definition you should say first: Staging and MERGE; do not invent keys. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 190. What is slowly changing dimension?

**1. Definition you should say first: Type 1 overwrite vs Type 2 history rows. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 191. How does a DBA support SCD2?

**1. Definition you should say first: PI, PPI on dates, and space for history. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 192. What is a surrogate key?

**1. Definition you should say first: Internal integer key; pick a PI that does not skew. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then**

**an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

### 193. What is a natural key?

**1. Definition you should say first: Business key such as account number. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

### 194. What is 3NF vs star schema?

**1. Definition you should say first: 3NF is normalized EDW; star is dimensional mart. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

### 195. Where does Teradata sit in a lakehouse?

**1. Definition you should say first: Often the governed EDW; NOS can query the lake. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

### 196. What is data lineage?

**1. Definition you should say first: Where a column came from; DBA supports via views and docs. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 197. What is a view vs a table?

**1. Definition you should say first: View is stored SQL; table is stored rows. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 198. What is a recursive view?

**1. Definition you should say first: View that refers to itself; use carefully. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.**

**3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.**

**4. Working code / syntax (write this on Teradata SQL):**

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

**5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.**

## 199. What is a locking view?

**1. Definition you should say first: A view that includes LOCKING modifiers. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.**

**3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.**

**4. Working code / syntax (write this on Teradata SQL):**

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

**5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.**

## 200. LOCKING ROW FOR ACCESS?

**1. Definition you should say first: Allows dirty reads to reduce blocking. Then spend the rest of the answer on architecture,**

a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

## 201. When is ACCESS locking dangerous?

1. Definition you should say first: When the reader must see committed consistent numbers. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

## 202. What is a deadlock victim?

1. Definition you should say first: The session Teradata aborts to break a deadlock. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

## 203. How do you tune a product join?

1. Definition you should say first: Stats, better join keys, or rewrite to merge/hash. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

## 204. What is a character set issue?

1. Definition you should say first: Latin vs Unicode; conversions can cost CPU. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table

**or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

## 205. What is a case-specific compare?

**1. Definition you should say first: CS vs CASESPECIFIC in comparisons. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

## 206. What is a format in BTEQ?

**1. Definition you should say first: How numbers and dates print in reports. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

## 207. What is .IF ERRORCODE in BTEQ?

**1. Definition you should say first: Branch on failure in a script. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
```

```
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

### 208. What is a return code in jobs?

**1. Definition you should say first: Scheduler uses it to fail the box. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

### 209. How do you alert on a failed load?

**1. Definition you should say first: Scheduler plus Viewpoint alerts plus mail. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

### 210. What is a runbook?

**1. Definition you should say first: Step-by-step for a repeat incident. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

### 211. What is a CAB?

**1. Definition you should say first: Change advisory board for production changes. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**

## 212. What is a back-out plan?

**1. Definition you should say first: How you undo a change if it fails. Then spend the rest of the answer on architecture, a working example, and a production failure.**

**2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.**

**3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.**

**4. Working code / syntax (write this on Teradata SQL):**

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

**5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.**